

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1. ОБЗОР ТЕКУЩЕГО СОСТОЯНИЯ ПРЕДМЕТА ИССЛЕДОВАНИЯ	7
1.1 Зарубежные исследования и глобальные тренды	7
1.2 Отечественные исследования и специфика рынка РФ	9
1.3 Общая характеристика проблемы исследования	11
2. ОБЗОР ТЕХНОЛОГИЙ И МЕТОДОВ РЕШЕНИЯ ПРОБЛЕМЫ	13
2.1 Основные методы решения проблемы и их характеристики	13
2.1.1 Анализ методов генерации: Импортонезависимость и RAG	13
2.1.2. Метод семантического поиска (GigaChat Embeddings)	14
2.2 Основные технологии решения проблемы и их характеристики	14
2.2.1. Серверная платформа: Сравнительный анализ Go и Python	14
2.2.2. Лингвистическая платформа: GigaChat и GigaChain	15
2.2.3. Стратегия хранения данных: Polyglot Persistence vs Single Store	16
2.3 Дополнительные технологии решения проблемы и их характеристики	16
2.3.1. Клиентская часть (Frontend)	16
2.3.2. Инструменты оркестрации	17
2.4 Алгоритм выбора методов и технологий решения проблемы	18
3. ФОРМИРОВАНИЕ ТРЕБОВАНИЙ К РЕШЕНИЮ ПРОБЛЕМЫ	19
3.1. Общая характеристика предлагаемого решения	19
3.2. Функциональные требования к предлагаемому решению	22
3.3. Нефункциональные требования к предлагаемому решению	23
3.4. Архитектурная модель системы	24
ЗАКЛЮЧЕНИЕ	30
Список использованных источников	31

ВВЕДЕНИЕ

Развитие цифровых экосистем и сервисов бронирования приводит к трансформации паттернов потребительского поведения в индустрии досуга и туризма. Современные платформы-агрегаторы ориентированы на предоставление максимального количества вариантов (сотни ресторанов, десятки развлекательных событий), которые отличаются по локации, ценовой категории и рейтинговым показателям.

Важной проблемой становится когнитивная перегрузка пользователя, возникающая при обработке избыточных массивов данных. Традиционные методы фильтрации контента («фасетный поиск») оказываются недостаточно эффективными в условиях, когда пользователю требуется не просто выборка по параметрам, а персонализированная рекомендация, учитывающая скрытый контекст (настроение, состав группы, временные ограничения). Одним из основных направлений развития в данной области является создание интеллектуальных диалоговых агентов (Conversational AI), способных брать на себя функцию предварительного отбора (кураторства) и сужения воронки выбора.

Актуальность исследования обусловлена противоречием между стремительным ростом количества предложений на рынке досуга и ограниченными психофизиологическими ресурсами человека для принятия решений. Значительная часть пользователей сталкивается с феноменом «парадокса выбора» (Choice Overload), когда изобилие альтернатив ведет не к удовлетворенности, а к отказу от совершения действия или снижению субъективной оценки результата [1]. В связи с этим возникает необходимость разработки технологий, обеспечивающих интеллектуальное ограничение выдачи (Shortlist) на основе анализа предпочтений пользователя.

Объектом исследования являются рекомендательные системы и диалоговые интерфейсы как инструмент для организации досуга и планирования времени.

Предмет исследования — методы и алгоритмы персонализированного поиска, направленные на снижение когнитивной нагрузки пользователя посредством сужения выборки релевантных альтернатив.

Цель данной научно-исследовательской работы заключается в теоретическом анализе проблемы избыточного выбора в цифровых сервисах и разработке требований к веб-приложению «Search My Way», предназначенному для интеллектуального подбора досуга с использованием механик свайп-голосования и AI-ассистента.

Для достижения поставленной цели необходимо решить следующие задачи:

- провести обзор отечественных и зарубежных публикаций, посвященных психологии потребительского выбора, трендам TravelTech и применению генеративного ИИ в сервисах бронирования;
- выполнить сравнительный анализ современных агрегаторов и рекомендательных сервисов, выявив их функциональные ограничения;
- сформировать функциональные и нефункциональные требования к проектируемому приложению, включая сценарии группового выбора;
- определить архитектурные особенности предлагаемого решения, включая интеграцию с внешними API и алгоритмы ранжирования выдачи.

Проблема исследования заключается в том, что существующие агрегаторы позволяют осуществлять поиск по формальным критериям, однако

они лишены механизмов контекстного понимания запроса и инструментов для совместного принятия решений группой пользователей. В условиях отсутствия «умного ограничения» наблюдается повышенная вероятность неудовлетворенности пользователя, известная как «усталость от принятия решений» (decision fatigue) [1].

Теоретическая значимость исследования заключается в систематизации подходов к проблеме Choice Overload в цифровой среде и обосновании эффективности модели «Smart Shortlist» (умный короткий список) по сравнению с моделью «Infinite Scroll» (бесконечная лента).

Практическая значимость заключается в создании методической основы для реализации полнофункционального программного продукта, ориентированного на повышение конверсии в бронирование и улучшение пользовательского опыта (UX) за счет делегирования рутины поиска алгоритмам.

Структура отчёта соответствует требованиям методических рекомендаций и включает введение, три аналитических раздела, заключение и список использованных источников.

В первой главе рассматриваются теоретические основы феномена избыточного выбора и обзор рынка.

Вторая глава посвящена анализу технологий реализации рекомендательных систем.

Третья глава ориентирована на формирование требований к программному решению и описание его архитектуры.

1. ОБЗОР ТЕКУЩЕГО СОСТОЯНИЯ ПРЕДМЕТА ИССЛЕДОВАНИЯ

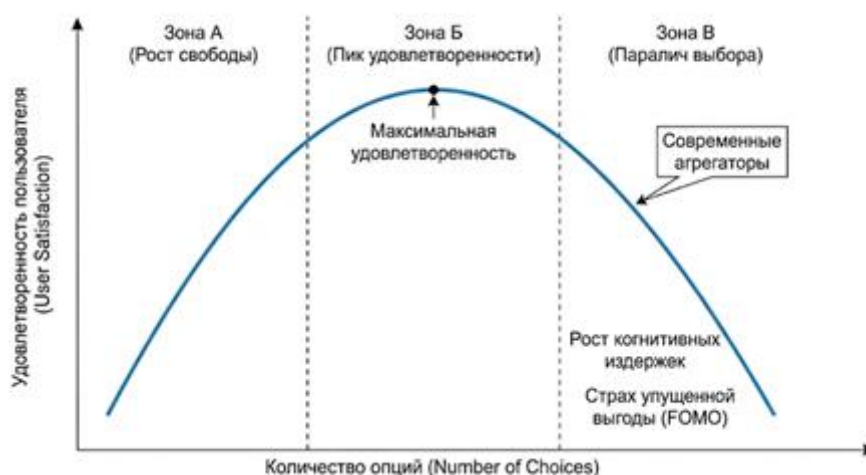
1.1 Зарубежные исследования и глобальные тренды

Современные зарубежные исследования в области электронной коммерции и психологии потребления характеризуются пересмотром парадигмы «чем больше выбор, тем лучше». Ключевым теоретическим базисом является концепция «парадокса выбора», сформулированная социологом Барри Шварцем. Согласно аналитическим публикациям и монографиям, экспоненциальный рост вариантов ведет к снижению мотивации пользователя.

Одним из наиболее заметных теоретических положений является утверждение Шварца о нелинейной зависимости благополучия от свободы выбора. В зарубежных работах подчеркивается, что при пересечении определенного порога изобилие начинает играть деструктивную роль. Как отмечает Б. Шварц: «По мере роста числа вариантов... позитивные аспекты сменяются негативными. По мере дальнейшего роста числа вариантов негативные эффекты нарастают до тех пор, пока мы не становимся перегруженными. В этой точке выбор больше не освобождает, а ослабляет» [1].

Для наглядной демонстрации зависимости между количеством альтернатив и уровнем удовлетворенности пользователя целесообразно привести графическую интерпретацию модели Б. Шварца (Рисунок 1.1).

Рисунок 1.1 — График зависимости удовлетворенности от количества опций



Как видно из графика, функция полезности имеет форму перевернутой параболы. На начальном этапе (зона А) увеличение выбора воспринимается позитивно, так как дает ощущение свободы. Однако после прохождения точки экстремума (зона Б — «Пик удовлетворенности») каждая новая добавленная опция начинает снижать субъективную ценность выбора из-за роста когнитивных издержек на сравнение и страха упущенной выгоды (FOMO). Большинство современных агрегаторов находятся в правой части графика (зона В), где эффективность взаимодействия критически падает [1]. Наша система стремится вернуть пользователя в зону Б за счет искусственного ограничения выдачи.

Эмпирическим подтверждением данной теории служат исследования поведения потребителей (Iyengar & Lepper), которые демонстрируют прямую корреляцию между сокращением ассортимента и ростом конверсии. В классическом эксперименте сокращение набора альтернатив с 24 до 6 привело к десятикратному росту покупок (с 3 % до 30 %) [1]. Это создает основу для пересмотра UX/UI стратегий глобальных сервисов.

В ответ на эти вызовы индустрия TravelTech внедряет новые стандарты взаимодействия. Согласно отчету Booking.com "Travel Predictions 2024", наблюдается фундаментальный сдвиг в сторону делегирования планирования

искусственному интеллекту [2]. 48 % пользователей готовы доверить ИИ построение маршрутов, чтобы избежать рутинной работы по сравнению десятков вкладок браузера.

Особое внимание уделяется гиперперсонализации. Исследования McKinsey & Company показывают, что 71 % потребителей ожидают от сервисов предиктивного поведения — предложения вариантов, основанных на их прошлых действиях и текущем контексте, а не просто выдачи каталога. Компании, внедряющие такие подходы, демонстрируют рост выручки на 40 % [3].

Таким образом, анализ зарубежных источников позволяет выделить следующие устойчивые направления развития сервисов досуга:

- переход от поисковых строк к диалоговым интерфейсам (Conversational AI);
- использование алгоритмов для искусственного ограничения выдачи
- интеграция психографических параметров (настроение, стиль отдыха) в алгоритмы ранжирования.

Данные тенденции подтверждают высокий интерес индустрии к созданию систем, способных не только агрегировать данные, но и брать на себя ответственность за первичный отсев нерелевантных вариантов.

1.2 Отечественные исследования и специфика рынка РФ

В России рынок цифровых сервисов для организации досуга находится на этапе зрелости технологий агрегации, но активного развития рекомендательных алгоритмов. Для отечественных исследований и продуктов характерна высокая степень централизации данных в рамках крупных экосистем.

Согласно данным Mediascope (2024), охват сервисов компании Яндекс составляет 83 % населения, что делает платформы «Яндекс.Карты» и «Яндекс.Еда» доминирующими игроками [4]. В работах российских аналитиков отмечается, что данные сервисы эффективно решают задачу навигации и транзакций, однако их рекомендательный потенциал ограничен моделью «геопозиция + рейтинг».

Функциональный анализ ключевых отечественных игроков выявляет следующие особенности:

- Яндекс.Карты / 2ГИС: ориентированы на работу с каталогом. Пользователь вынужден самостоятельно настраивать множество фильтров. Рекомендации строятся на массовой популярности («Топ мест»), что часто не соответствует индивидуальному запросу «здесь и сейчас»;
- KudaGo: специализируется на событийном контенте. Основной формат потребления — просмотр редакционных подборок («Куда пойти в выходные»). Это снижает усилия на поиск, но не решает проблему персонализации, так как подборки статичны и одинаковы для всех;
- локальные стартапы (например, «Кавёр»): предпринимают попытки внедрения свайп-механик, однако часто ограничены узкой географией (Москва, Санкт-Петербург) и отсутствием инструментов для анализа групповых предпочтений.

Для систематизации недостатков существующих решений был проведен функциональный бенчмаркинг ключевых игроков рынка. Результаты сравнительного анализа представлены в Таблице 1.1.

Таблица 1.1 — Матрица функционального сравнения сервисов досуга

Функция / Критерий	Яндекс.Карты / 2ГИС	KudaGo	TripAdvisor	Search My Way (Разрабатываемое)
--------------------	---------------------	--------	-------------	---------------------------------

Основной сценарий	Навигация и поиск конкретного места	Просмотр афиши событий	Чтение отзывов туристов	Интеллектуальный подбор
Учет контекста (ИИ)	Нет (только фильтры)	Нет (статичные подборки)	Нет	Есть (GigaChat RAG)
Групповое голосование	Нет (пересылка ссылок)	Нет	Нет	Есть (Real-time Lobby)
Свайп-механика	Нет	Нет	Нет	Есть
Персонализация	История поиска	История просмотров	История оценок	Векторный профиль интересов

На основе данной матрицы можно сделать вывод, что ни один из прямых конкурентов не закрывает потребность в **социальной координации** (групповой выбор) и **диалоговом поиске**, что подтверждает наличие свободной рыночной ниши для разрабатываемого продукта.

В отечественной практике остается нерешенной проблема группового выбора. Для молодежного сегмента (18–25 лет) критически важен процесс согласования места встречи. В текущих реалиях это происходит в мессенджерах, что приводит к дезорганизации и конфликтам. На российском рынке практически отсутствуют массовые инструменты, автоматизирующие компромисс через игровые механики.

Актуальные направления развития отечественных сервисов включают создание суперприложений, однако ниша специализированных AI-помощников для досуга, работающих по принципу «консьерж-сервиса», остается свободной.

1.3 Общая характеристика проблемы исследования

Анализ зарубежных трендов и отечественного рынка позволяет сформулировать основные проблемы, препятствующие эффективной реализации пользовательских сценариев при поиске досуга.

Во-первых, наблюдается технологический разрыв между ожиданиями пользователей и функционалом интерфейсов. С одной стороны, 89 %

пользователей готовы использовать ИИ для упрощения жизни [2]. С другой стороны, существующие агрегаторы предлагают интерфейсы десятилетней давности, заставляющие человека выполнять роль «фильтра» данных.

Во-вторых, проблема когнитивной нагрузки («Парадокс выбора») игнорируется большинством сервисов в угоду рекламной модели монетизации, которая требует показа максимального количества позиций. Это приводит к снижению лояльности аудитории и увеличению времени принятия решения (Time-to-Decision).

В-третьих, существующие решения не учитывают динамический контекст. Потребности пользователя в пятницу вечером (с друзьями) и в понедельник днем (бизнес-ланч) кардинально отличаются, однако традиционные алгоритмы часто предлагают одни и те же заведения, основываясь только на истории посещений, без возможности уточнения «намерений» через диалог.

Кроме того, остается нерешенной задача социальной координации. Процесс выбора места для группы людей является стрессовым фактором, и отсутствие инструментов для «слепого голосования» или автоматического подбора пересечений интересов снижает частоту совместных выходов.

Таким образом, разработка приложения «Search My Way» является ответом на подтвержденный рыночный запрос на снижение информационной перегрузки и автоматизацию рутины выбора. Решение проблемы требует перехода от модели «Поиск» (Search) к модели «Рекомендация и Действие» (Recommend & Act).

2. ОБЗОР ТЕХНОЛОГИЙ И МЕТОДОВ РЕШЕНИЯ ПРОБЛЕМЫ

2.1 Основные методы решения проблемы и их характеристики

Проектирование архитектуры интеллектуальной системы поиска досуга требует анализа существующих методологических подходов к обработке естественного языка и ранжированию данных. В рамках исследования был проведен сравнительный анализ методов извлечения контекста и генерации ответов.

2.1.1 Анализ методов генерации: Импортонезависимость и RAG [17]

Для обеспечения устойчивости бизнеса и соблюдения законодательства РФ (152-ФЗ) [15] критически важен выбор провайдера LLM. Рассматривались два подхода: использование зарубежных API (OpenAI) и отечественных решений (GigaChat).

- **RAG с зарубежными LLM:** обеспечивает высокое качество, но несет риски блокировки аккаунтов, сложности с оплатой (валютные риски) и проблемы с передачей персональных данных за границу.
- **RAG с GigaChat:** позволяет реализовать генерацию ответов с использованием серверов, находящихся в РФ. Модель GigaChat нативно обучена на русскоязычном корпусе данных, что обеспечивает лучшее понимание культурного кода (сленг, названия локальных блюд) по сравнению с англоцентричными моделями.

Решение: Выбран метод RAG на базе GigaChat. Это минимизирует регуляторные риски и обеспечивает нативное качество генерации для русскоязычных пользователей.

2.1.2. Метод семантического поиска (GigaChat Embeddings)

Для работы RAG необходима векторизация запросов. Использование GigaChat Embeddings позволяет проецировать запросы пользователя и описания мест в единое семантическое пространство[5], [6], оптимизированное под русский синтаксис. Это решает проблему «языкового разрыва», характерную для моделей, обученных преимущественно на английском языке (например, text-embedding-ada-002).

2.2 Основные технологии решения проблемы и их характеристики

Выбор технологического стека базировался на критериях высокой конкурентности (High Concurrency), строгой согласованности данных и минимизации инфраструктурной сложности.

2.2.1. Серверная платформа: Сравнительный анализ Go и Python

Для реализации микросервиса оркестрации (Backend) рассматривались языки Python и Go.

Таблица 2.1 – Сравнительный анализ серверных технологий

Характеристика	Python (FastAPI / Django)	Go (Gin / Echo)	Влияние на проект «Search My Way»
Модель конкурентности	Потоки ОС / Event Loop (GIL ограничивает параллелизм).	Горутины (Goroutines). Легковесные потоки, управляемые рантаймом.	Go позволяет держать 10k+ активных WebSocket-сессий на одном узле (важно для лобби).
Производительность	Интерпретируемый (медленный, требует оптимизации).	Компилируемый (близок к C++).	Снижение задержки (Latency) в RAG-пайплайне.
Типизация	Динамическая (риск ошибок в рантайме).	Строгая статическая.	Повышение надежности финансово-значимого кода.

Экосистема ML	Доминирующая (PyTorch, TensorFlow).	Ограниченная.	Т.к. инференс ML вынесен в API/Контейнеры, преимущество Python нивелируется.
---------------	-------------------------------------	---------------	--

Решение: Выбран язык **Go** [7], так как архитектура системы предполагает вынос тяжелых вычислений (ML) во внешние сервисы, оставляя бэкенду роль высокопроизводительного I/O-роутера.

2.2.2. Лингвистическая платформа: GigaChat и GigaChain

В качестве основного интеллекта системы используется GigaChat (Sber) [13]. Интеграция выполняется через библиотеку GigaChain [14] (адаптация LangChain для экосистемы Сбера).

Таблица 2.2 – Сравнительный анализ LLM-провайдеров для рынка РФ

Критерий сравнения	OpenAI (GPT-4)	GigaChat (Sber)	Аналитический вывод
Локализация данных	США / Европа	Россия (ЦОД Сбера)	GigaChat соответствует требованиям 152-ФЗ «О персональных данных».
Оплата услуг	USD (сложности с картами РФ)	RUB (прямой договор)	Использование GigaChat исключает валютные риски и проблемы с транзакциями.
Понимание языка	Высокое, но возможны калки с английского	Нативное (Русский язык)	GigaChat лучше понимает локальный контекст (напр., «шаверма» vs «шаурма»).
Экосистема	Закрытая	Интеграция сервисами РФ	Доступ к SaluteSpeech (синтез речи) и GigaChain из коробки.

Решение: Переход на **GigaChat** является стратегическим решением для обеспечения юридической чистоты и стабильности сервиса.

2.2.3. Стратегия хранения данных: Polyglot Persistence vs Single Store

Критическим архитектурным решением стал выбор места хранения векторных эмбеддингов.

Таблица 2.3 – Сравнительный анализ архитектур хранения векторов

Критерий	Специализированная Векторная БД (Qdrant)	Расширение для Реляционной БД (PostgreSQL + pgvector)	Решение аналитика
Архитектура	Отдельный сервис (Polyglot).	Единый сервис (Single Store).	pgvector упрощает администрирование.
Согласованность данных	Eventual Consistency (нужна синхронизация ID).	ACID . Векторы и метаданные обновляются в одной транзакции.	pgvector исключает рассинхрон данных.
Сложность запросов	Ограничена фильтрацией по payload.	SQL JOIN . Возможность объединять векторный поиск с гео-запросами и бизнес-логикой.	Возможность сложных выборок (Гео + Вектор + Цена).
Производительность	Экстремальная (миллиарды векторов).	Высокая (до 10-50 млн векторов).	Для задачи MVP производительность pgvector избыточна.

Решение: Выбран стек **PostgreSQL + pgvector**. Отказ от внедрения лишнего компонента (специализированной БД) снижает риски рассогласования данных и упрощает бэкапы.

2.3 Дополнительные технологии решения проблемы и их характеристики

2.3.1. Клиентская часть (Frontend)

Для разработки мобильного приложения выбран фреймворк Flutter. Ниже представлено сравнение с ключевым конкурентом — React Native.

Таблица 2.4 – Сравнительный анализ фреймворков мобильной разработки

Параметр	React Native	Flutter	Преимущество для проекта
Графический движок	Использует нативные UI-компоненты через JS-Bridge.	Skia / Impeller . Рисует каждый пиксель самостоятельно.	Flutter гарантирует идентичный UI на всех версиях iOS/Android (Pixel Perfect).
Производительность	Зависит от моста (Bridge). Возможны просадки FPS.	Нативная (компиляция в ARM-код).	Стабильные 60–120 FPS критичны для плавности свайп-механики.
Стабильность	Зависит от обновлений платформы OEM.	Независим от платформы.	Снижение затрат на поддержку старых устройств.

Решение: мобильное приложение разрабатывается на фреймворке **Flutter** [11]. Использование движка Impeller обеспечивает плавность анимаций, критичную для UX.

2.3.2. Инструменты оркестрации

Для управления промптами и контекстом диалога используется GigaChain (SDK на базе LangChain). Это позволяет использовать готовые абстракции (Chains, Memory, Retrievers), адаптированные для работы с API Сбера.

Таблица 2.5 – Сводная матрица технологического стека

Компонент	Компонент	Роль в системе
Backend	Go (Gin)	Высокопроизводительный API Gateway и управление WebSocket-сессиями.
AI Provider	GigaChat	Генерация ответов, векторизация текста (Embeddings), классификация интенгов.
Database	PostgreSQL	Хранение реляционных данных (пользователи, места) и векторов (pgvector).
Frontend	Flutter	Кроссплатформенный интерфейс с нативной производительностью.

2.4 Алгоритм выбора методов и технологий решения проблемы

Выбор архитектуры (Go + GigaChat + Postgres + Flutter) обоснован многокритериальным анализом рисков и эффективности:

1. **Юридический суверенитет (Compliance):** Использование **GigaChat** снимает риски, связанные с трансграничной передачей данных. Сервера находятся в России, что автоматически закрывает вопросы соответствия 152-ФЗ, критичные для любого B2C-сервиса, обрабатывающего предпочтение пользователей.

2. **Экономическая эффективность (TCO):** Использование **PostgreSQL + pgvector** вместо платных облачных векторных БД (типа Pinecone) и оплата токенов GigaChat в рублях по фиксированным тарифам позволяют прогнозировать операционные расходы (OPEX) и избегать курсовой волатильности.

3. **Технологическая синергия:** Связка **GigaChain + GigaChat** предоставляет оптимизированные пайплайны для RAG (Retrieval-Augmented Generation), что снижает трудозатраты на разработку («time-to-market») по сравнению с написанием собственных адаптеров для OpenAI.

4. **Производительность:** Язык **Go** обеспечивает необходимый запас производительности для микросервисной архитектуры, позволяя масштабировать систему без переписывания кода, в отличие от Python-решений, упирающихся в GIL при высоких нагрузках.

Таким образом, выбранный стек является оптимальным решением для создания устойчивого, легального и производительного сервиса на российском рынке.

3. ФОРМИРОВАНИЕ ТРЕБОВАНИЙ К РЕШЕНИЮ ПРОБЛЕМЫ

3.1. Общая характеристика предлагаемого решения

Предлагаемое программное решение «Search My Way» представляет собой кроссплатформенную интеллектуальную систему для организации досуга, реализующую гибридную архитектуру рекомендаций. Система предназначена для автоматизации процесса выбора заведений (рестораны, кафе, культурные события) как в индивидуальном режиме, так и в формате группового голосования.

Технически решение базируется на архитектуре **RAG (Retrieval-Augmented Generation)**, адаптированной под российский технологический стек. Генеративная модель **GigaChat** дополняется актуальными данными о заведениях, хранящимися в локальной базе данных, что обеспечивает фактологическую точность рекомендаций и соответствие законодательству РФ.

Архитектура решения включает три логических уровня, взаимодействующих по сетевым протоколам:

1. **Серверная часть (Backend):** Реализована на языке **Go** с использованием фреймворка **Gin** [8]. Обеспечивает высокопроизводительную обработку HTTP-запросов и поддерживает постоянные WebSocket-соединения для реализации механики групповых комнат (Lobby) в реальном времени.

2. **Уровень данных (Data Layer):** базируется на СУБД **PostgreSQL** [9]. Для реализации семантического поиска используется расширение **pgvector**, позволяющее хранить векторные представления (эмбединги) описаний мест в едином контуре с реляционными данными. Геопространственная фильтрация осуществляется средствами **PostGIS**.

3. **Клиентская часть (Frontend):** разработана на фреймворке **Flutter**, что обеспечивает создание единого нативного приложения для ОС Android и iOS. Интерфейс включает модуль диалога с ИИ-ассистентом и модуль карточек для свайп-голосования.

Особое внимание уделено интеграции с лингвистической платформой Сбера. Система не просто передает запрос пользователя в модель, а предварительно обогащает его контекстом через библиотеку **GigaChain**: историей предпочтений, текущей геолокацией и списком релевантных заведений, найденных через векторный поиск.

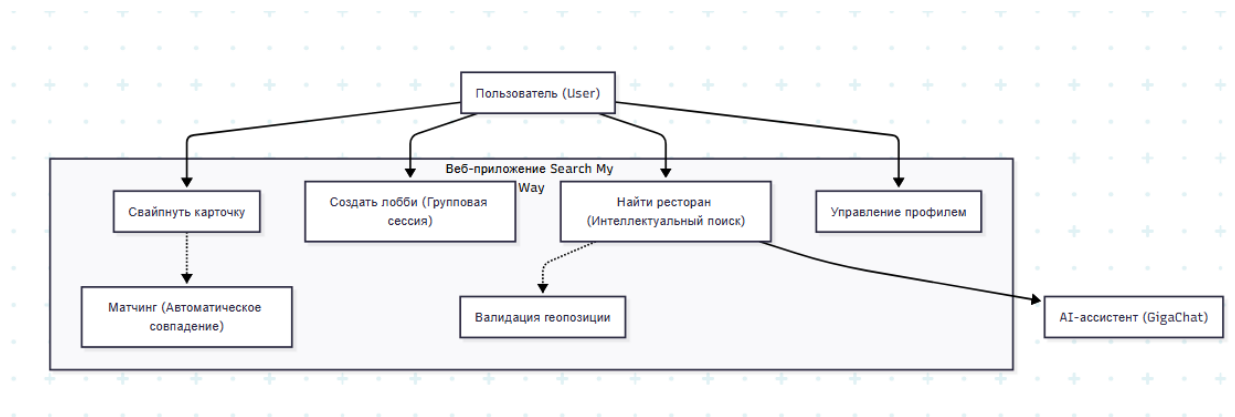
Ключевыми функциональными особенностями системы являются:

- семантический анализ запросов на русском языке с учетом локального контекста;
- гибридное ранжирование объектов (геопозиция + смысловая близость);
- синхронизация действий пользователей в режиме группового выбора (Real-time);
- генерация персонализированных обоснований рекомендаций («Почему это место подходит вам»).

Для формализации функциональных границ системы была разработана диаграмма вариантов использования (Use Case Diagram), представленная на

Рисунке 3.1. Она определяет взаимодействия основных действующих лиц с системой.

Рисунок 3.1 — Диаграмма вариантов использования (Use Case)



В системе выделены два ключевых актора:

1. **Пользователь (User):** Инициатор запросов. Имеет возможность управлять личным профилем, инициировать одиночный поиск через чат и создавать групповые комнаты.
2. **AI-ассистент (GigaChat System):** Внешний актор, обеспечивающий генерацию ответов.

Ключевые прецеденты (Use Cases):

- **«Интеллектуальный поиск»:** включает в себя сценарии ввода запроса, валидации геопозиции и получения ранжированного списка.
- **«Групповая сессия»:** сложный сценарий, включающий создание уникальной ссылки, подключение участников через WebSocket и синхронизацию состояний голосования.
- **«Матчинг»:** автоматический процесс выявления совпадений, который запускается системой при наличии лайков на одном объекте от всех участников лобби.

3.2. Функциональные требования к предлагаемому решению

Функциональные требования определяют перечень операций, которые система должна выполнять [20] для обеспечения потребностей пользователей. Они сформированы на основе анализа пользовательских сценариев (Use Cases), выявленных в аналитической части исследования.

Таблица 3.1 – Функциональные требования

Требование	Описание
Авторизация и профилирование	Система должна обеспечивать регистрацию пользователей (Sber ID, телефон), хранение их устойчивых предпочтений (любимые кухни, аллергии, средний чек) и истории посещений в защищенном контуре.
Семантический гео-поиск	Система должна обрабатывать естественно-языковые запросы (например, «где поесть рамен недалеко от центра») и ранжировать результаты на основе векторной близости (GigaChat Embeddings) и удаленности пользователя (PostGIS).
AI-консультант (RAG)	Система должна поддерживать диалог с пользователем, отвечая на уточняющие вопросы о меню, атмосфере и ценах, используя контекст, извлеченный из базы данных, для минимизации галлюцинаций.
Управление Лобби (Group Lobby)	Система должна позволять создавать виртуальные «комнаты» для группы пользователей, генерировать deep-link ссылки для приглашения и синхронизировать статус участников через WebSocket.
Коллективное голосование	Система должна реализовывать механику «свайпов» (лайк/дизлайк) для списка заведений. Алгоритм должен в реальном времени определять совпадение (Match) и уведомлять всех участников.
Генерация резюме (Summary)	Система должна отображать карточку заведения с фото, рейтингом и сгенерированным GigaChat кратким резюме, объясняющим, почему это место рекомендовано конкретной группе.

Совокупность зафиксированных требований покрывает полный цикл User Journey: от возникновения нечеткой потребности до принятия коллективного решения. Реализация механизма группового голосования на базе WebSocket и Go является уникальным функциональным преимуществом (USP) по сравнению с традиционными агрегаторами-каталогами.

3.3. Нефункциональные требования к предлагаемому решению

Нефункциональные требования определяют качественные атрибуты системы [18]: производительность, надежность, безопасность и соответствие регуляторным нормам РФ.

Таблица 3.2 – Нефункциональные требования

Категория	Требование
Производительность (Latency)	Время ответа API на поисковый запрос (включая RAG-генерацию) не должно превышать 1,5–2 сек. Синхронизация свайпа в групповом режиме должна происходить с задержкой не более 100 мс (Real-time).
Масштабируемость	Архитектура должна поддерживать горизонтальное масштабирование серверной части (Stateless Go-containers) [16] для обслуживания до 10 000 одновременных пользователей без деградации сервиса.
Кроссплатформенность	Клиентское приложение на Flutter должно обеспечивать идентичное отображение интерфейса и функциональность (Pixel Perfect) на ОС Android (версии 10+) и iOS (версии 15+).
Соответствие 152-ФЗ	Хранение и обработка персональных данных должны осуществляться на серверах, расположенных на территории РФ. Взаимодействие с AI-провайдером (GigaChat) должно происходить по защищенным каналам без трансграничной передачи данных.
Эффективность ресурсов	Размер установочного файла мобильного приложения (APK/IPA) не должен превышать 100 МБ. Потребление памяти на

	сервере (Go Runtime) должно быть оптимизировано для работы в контейнерах с лимитом 512 МБ RAM.
Доступность (Availability)	Коэффициент доступности сервиса должен составлять не менее 99,9% (Uptime). Система должна корректно обрабатывать обрывы связи мобильной сети (Reconnection policy).

Соблюдение данных требований обеспечивается выбором компилируемого языка Go для бэкенда, отечественной LLM GigaChat и технологии Flutter для фронтенда.

3.4. Архитектурная модель системы

Архитектурная модель системы строится на принципах **Clean Architecture** («Чистая архитектура») с элементами микросервисного подхода. Это обеспечивает разделение ответственности, тестируемость и независимость бизнес-логики от внешних фреймворков и API[19].

В основе системы лежит клиент-серверное взаимодействие, разделенное на логические слои:

1. Уровень представления (Presentation Layer):

Мобильное приложение на Flutter. Реализует паттерн BLoC (Business Logic Component) [12]. Отвечает за отрисовку UI, анимации свайпов (используя движок Impeller) и локальное кеширование данных.

Взаимодействие с сервером реализуется по гибридной модели:

- **REST API (HTTP/1.1):** используется для операций, не требующих постоянного соединения (аутентификация, загрузка профиля, получение истории поиска), что позволяет экономить ресурсы сервера.

- **WebSocket (WSS):** используется для обеспечения двусторонней связи в реальном времени (поточная передача ответов GigaChat, синхронизация статусов участников в групповом лобби).

2. Транспортный уровень (Gateway Layer):

Реализован на Go с использованием фреймворка Gin. Принимает запросы, валидирует входные данные (JSON Schema), проверяет токены авторизации (JWT Middleware) и маршрутизирует запросы к сервисам ядра.

3. Уровень бизнес-логики (Domain Layer):

Ядро системы. Здесь находятся основные сервисы:

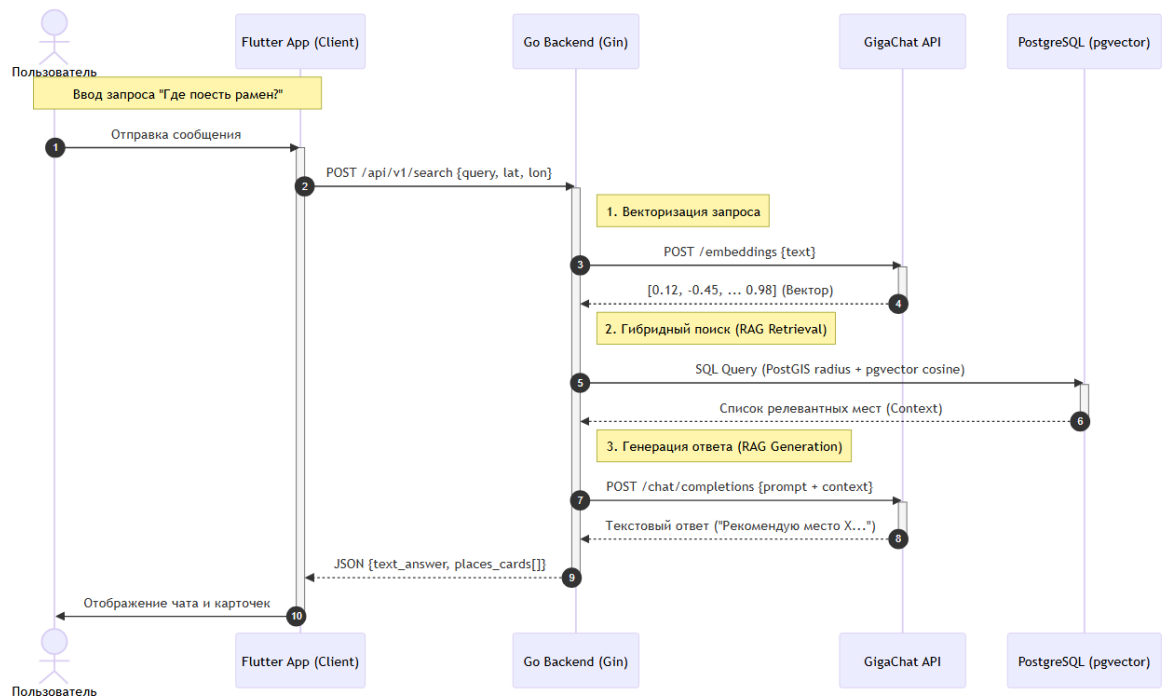
- **SearchService:** управляет логикой ранжирования и RAG-пайплайном (интеграция с GigaChain).
- **LobbyService:** управляет состоянием групповых комнат (State Machine).
- **AuthService:** управляет сессиями пользователей.

4. Уровень инфраструктуры (Infrastructure Layer):

Реализует доступ к данным:

- **PostgreSQL + pgvector:** Единое хранилище для профилей, геоданных и векторов.
- **GigaChat API:** Клиент для генерации ответов и эмбедингов.

Рисунок 3.1 — Схема пайплайна обработки запроса



Представленная на *Рисунке 3.1* диаграмма последовательности демонстрирует поток данных в системе при обработке пользовательского интента. Процесс включает следующие этапы:

1. **Инициация запроса:** Пользователь вводит естественно-языковой запрос (например, «Где поесть ramen?») в интерфейсе мобильного приложения.

2. **Передача данных:** Клиентское приложение (Flutter App) формирует запрос к API Gateway (Go Backend), передавая текст и текущие координаты.

3. **Векторизация:** Сервер выделяет текстовую часть запроса и отправляет её в API GigaChat (эндпоинт /embeddings) для получения векторного представления.

4. **Гибридный поиск (Retrieval):** Бэкенд выполняет SQL-запрос к PostgreSQL, объединяя фильтрацию по геолокации (PostGIS) и ранжирование по косинусному сходству (pgvector) с полученным вектором. База данных возвращает список релевантных мест (Context).

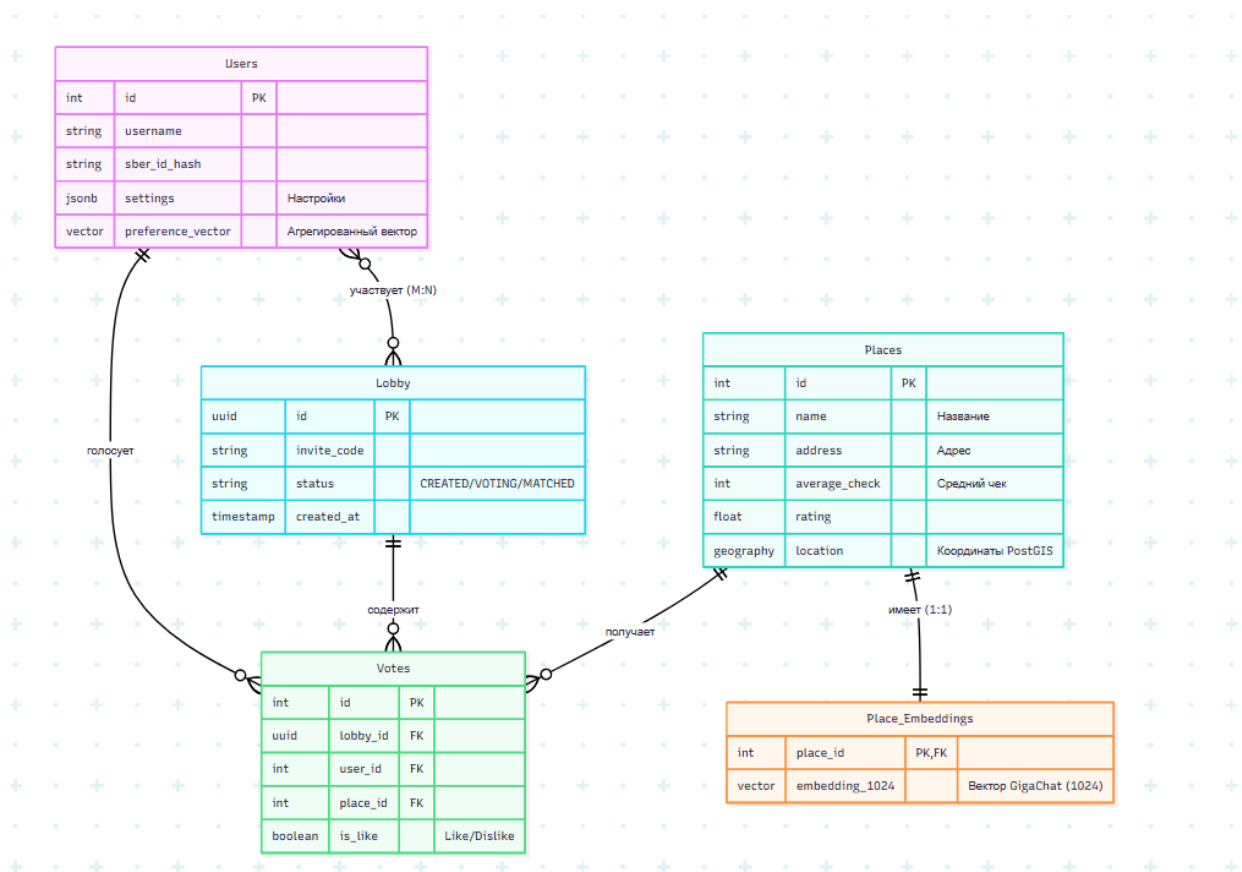
5. **Генерация ответа (Generation):** Сервер формирует системный промпт, объединяя вопрос пользователя и найденный контекст. Запрос отправляется в GigaChat (эндпоинт /chat/completions) для генерации финального ответа на естественном языке.

6. **Доставка результата:** Бэкенд объединяет текст ответа и структурированные карточки мест в JSON и возвращает его клиенту для отображения.

Данная архитектура обеспечивает гибкость и юридическую безопасность, так как все чувствительные данные обрабатываются внутри контролируемого контура, а внешние вызовы к GigaChat содержат только обезличенные поисковые запросы.

Проектирование уровня данных (Data Layer) выполнено с использованием реляционной модели. Инфологическая схема базы данных (ER-диаграмма) представлена на Рисунке 3.2.

Рисунок 3.2 — Схема базы данных (ER-Diagram)

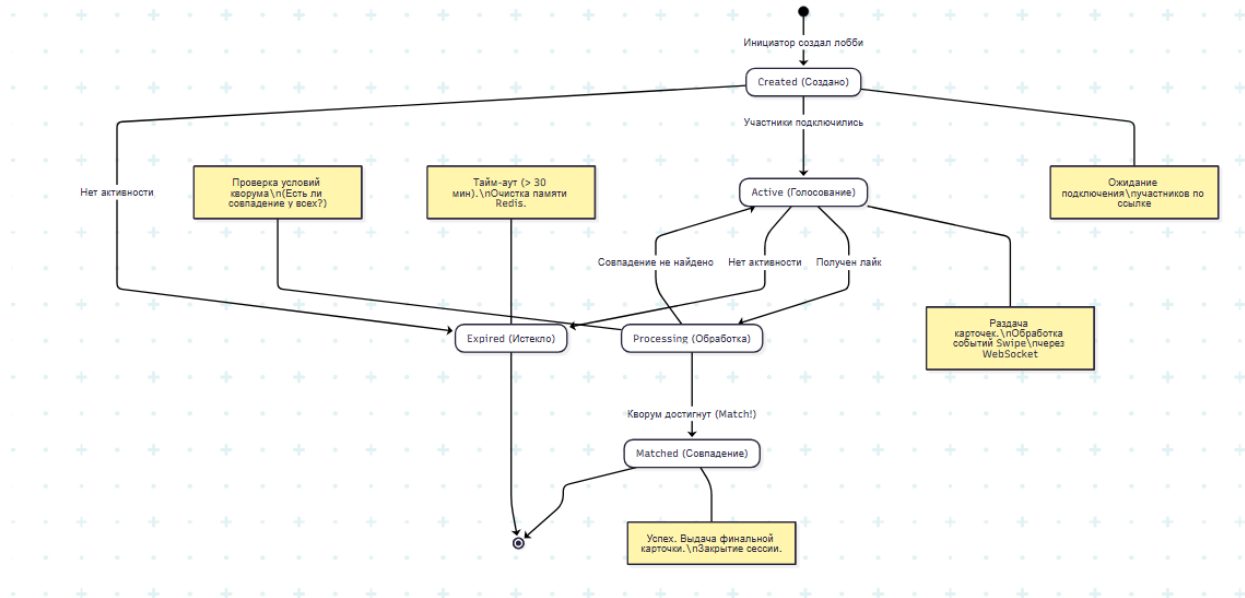


Описание основных сущностей системы:

1. **Users (Пользователи):** хранит учетные данные, настройки и агрегированный вектор предпочтений пользователя.
2. **Places (Места):** содержит статическую информацию (название, адрес, средний чек), полученную от картографических сервисов.
3. **Place_Embeddings (Векторы мест):** ключевая таблица для работы расширения pgvector. Хранит 1024-мерные векторы описаний мест, сгенерированные моделью GigaChat. Именно по этой таблице выполняется семантический поиск (KNN)[10].
4. **Lobby (Лобби):** временная сущность, хранящая состояние групповой комнаты (список участников, текущий статус).

Логика работы группового подбора описывается конечным автоматом (State Machine). Диаграмма состояний сущности «Лобби» приведена на Рисунке 3.3.

Рисунок 3.3 — Диаграмма состояний сессии подбора (State Machine Diagram)



Жизненный цикл сессии включает следующие состояния:

- **Created (Создано):** Инициатор получил ссылку, ожидает подключения других участников.
- **Active (Голосование):** Все участники подключились, система начала раздачу карточек заведений. В этом состоянии сервер принимает входящие события «Swipe» через WebSocket.
- **Processing (Обработка):** Система фиксирует потенциальное совпадение, проверяет условия кворума (все участники лайкнули одно место).
- **Matched (Совпадение):** Успешное завершение. Система выдает финальную карточку всем участникам и закрывает возможность голосования.

- **Expired (Истекло):** Состояние тайм-аута, если активность в комнате отсутствовала более 30 минут (очистка памяти Redis).

5. **Votes (Голоса):** таблица связи «Многие-ко-многим», фиксирующая реакции пользователей (Like/Dislike) на конкретные заведения в рамках сессии.

Такая нормализованная структура обеспечивает целостность данных и высокую скорость выполнения SQL-запросов с джойнами (JOIN), необходимых для гибридной фильтрации.

ЗАКЛЮЧЕНИЕ

В ходе выполнения научно-исследовательской работы была достигнута основная цель — проведен теоретический анализ проблемы избыточного выбора в цифровых сервисах и разработаны требования к веб-приложению «Search My Way», реализующему концепцию диалогового AI-ассистента с функцией группового подбора досуга.

В результате исследования были решены следующие задачи:

1. **Проанализирована теоретическая база.** На основе работ Б. Шварца и исследований поведения потребителей (Booking.com, McKinsey) подтверждена гипотеза о том, что избыточное количество вариантов (Choice Overload) снижает удовлетворенность пользователей. Обоснована необходимость перехода от парадигмы «Каталог» к парадигме «Умная рекомендация».

2. **Выявлена рыночная ниша.** Анализ отечественного рынка (Яндекс, KudaGo) показал отсутствие инструментов для социальной координации (Group Decision Support). Механика групповых «свайпов»,

предложенная в работе, является уникальным торговым предложением (USP), решающим проблему конфликтов при выборе места встречи.

3. **Обоснован технологический стек.** Сравнительный анализ подтвердил эффективность выбранной архитектуры:

- Язык **Go** обеспечивает высокую производительность для WebSocket-соединений в реальном времени.
- Модель **GigaChat** (Sber) гарантирует юридическую чистоту (152-ФЗ) и высокое качество генерации на русском языке.
- Стек **PostgreSQL + pgvector** позволяет реализовать гибридный поиск без усложнения инфраструктуры.
- Фреймворк **Flutter** обеспечивает кроссплатформенность и нативную плавность интерфейса.

4. **Сформированы требования и архитектура.** Разработаны функциональные требования, покрывающие полный пользовательский путь (CJM), и спроектирована RAG-архитектура, минимизирующая риск «галлюцинаций» нейросети за счет валидации данных через базу знаний.

Практическая значимость работы заключается в создании готовой спецификации для разработки MVP (Minimum Viable Product). Предложенное решение обладает потенциалом масштабирования и может быть внедрено как самостоятельный продукт или модуль в экосистемы отечественных TravelTech-компаний.

Список использованных источников

[1] Schwartz, B. The Paradox of Choice: Why More Is Less / B. Schwartz. — New York: Ecco, 2004. — 265 p.

[2] Travel Predictions 2024 [Электронный ресурс] // Booking.com: Newsroom. URL: <https://www.hospitalitynet.org/news/4118949.html> (дата обращения: 21.12.2025).

[3] The value of getting personalization right—or wrong—is multiplying [Электронный ресурс] // McKinsey & Company: Business Functions. URL: <https://www.mckinsey.com/capabilities/growth-marketing-and-sales/our-insights/the-value-of-getting-personalization-right-or-wrong-is-multiplying> (дата обращения: 21.12.2025).

[4] Обзор аудитории интернета в России [Электронный ресурс] // Mediascope: Библиотека данных. URL: <https://mediascope.net/data/> (дата обращения: 21.12.2025).

[5] Mikolov, T. Efficient Estimation of Word Representations in Vector Space / T. Mikolov et al. // arXiv preprint arXiv:1301.3781. — 2013.

[6] What is Cosine Similarity? [Электронный ресурс] // ScienceDirect: Topics. URL: <https://www.sciencedirect.com/topics/computer-science/cosine-similarity> (дата обращения: 21.12.2025).

[7] The Go Programming Language Specification [Электронный ресурс] // Go.dev. URL: <https://go.dev/ref/spec> (дата обращения: 21.12.2025).

[8] Gin Web Framework Documentation [Электронный ресурс] // Gin-gonic.com. URL: <https://gin-gonic.com/> (дата обращения: 21.12.2025).

[9] PostgreSQL 16 Documentation [Электронный ресурс] // PostgreSQL.org. URL: <https://www.postgresql.org/docs/16/index.html> (дата обращения: 21.12.2025).

[10] pgvector: Open-source vector similarity search for Postgres [Электронный ресурс] // GitHub. URL: <https://github.com/pgvector/pgvector> (дата обращения: 21.12.2025).

[11] Flutter Architectural Overview [Электронный ресурс] // Flutter.dev. URL: <https://docs.flutter.dev/resources/architectural-overview> (дата обращения: 21.12.2025).

[12] Bloc Library Documentation [Электронный ресурс] // Bloclibrary.dev. URL: <https://bloclibrary.dev/> (дата обращения: 21.12.2025).

[13] GigaChat API Documentation [Электронный ресурс] // Developers.sber.ru. URL: <https://developers.sber.ru/docs/ru/gigachat/api/reference> (дата обращения: 21.12.2025).

[14] GigaChain: LangChain for GigaChat [Электронный ресурс] // GitHub. URL: <https://github.com/ai-forever/gigachain> (дата обращения: 21.12.2025).

[15] Федеральный закон от 27.07.2006 N 152-ФЗ "О персональных данных" [Электронный ресурс] // КонсультантПлюс. URL: http://www.consultant.ru/document/cons_doc_LAW_61801/ (дата обращения: 21.12.2025).

[16] Docker Overview [Электронный ресурс] // Docker Docs. URL: <https://docs.docker.com/get-started/overview/> (дата обращения: 21.12.2025).

[17] Lewis, P. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks / P. Lewis et al. // Advances in Neural Information Processing Systems. — 2020. — Vol. 33. — P. 9459–9474.

[18] ISO/IEC 25010:2011. Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models. — Geneva: ISO, 2011. — 34 p.

[19] Martin, R. C. Clean Architecture: A Craftsman's Guide to Software Structure and Design / R. C. Martin. — London: Prentice Hall, 2017. — 432 p.

[20] Sommerville, I. Software Engineering / I. Sommerville. — 10th ed. — Boston: Pearson, 2015. — 816 p.